# PROJECT 2: WEB APPLICATION PENETRATION TESTING (OWASP TOP 10 FOCUS)


Submitted by:
Saikumar Joshi
Alex jose
Manas veetil
Afrin A N

[Grab your reader's attention with a great quote from the document or use this space to emphasize a key point. To place this text box anywhere on the page, just drag it.]

# DVWA Installation using TryHackMe

**1. Introduction**

The purpose of this setup was to prepare a safe and controlled environment for practicing penetration testing skills based on the **OWASP Top 10 vulnerabilities**. For this purpose, **Damn Vulnerable Web Application (DVWA)** was installed and accessed via the **TryHackMe platform**.

DVWA is intentionally designed to be insecure, providing multiple security levels to practice web exploitation techniques, such as SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and more.

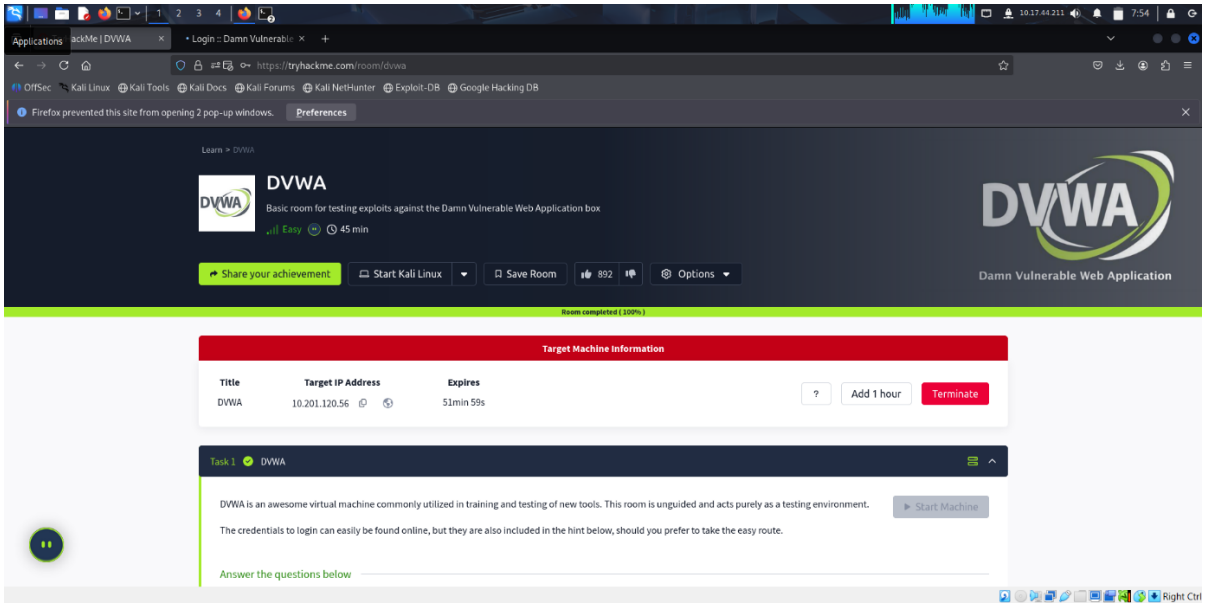
**2. Environment Setup**

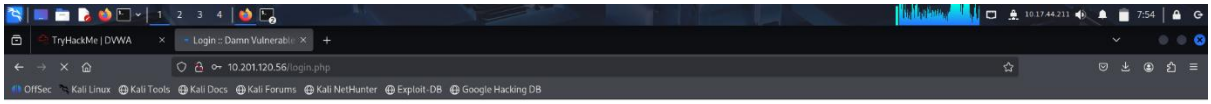
**2.1 Tools & Platforms Used**

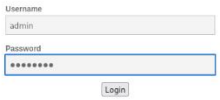
- **TryHackMe** – Online cybersecurity training platform providing pre-configured vulnerable labs.
- **DVWA (Damn Vulnerable Web Application)** – Target vulnerable application.
- **Kali Linux** (local/VM) – Attacker machine.
- **Burp Suite Community Edition** – Proxy tool for interception and exploitation.
- **Web Browser (Firefox/Chrome)** – To interact with DVWA.

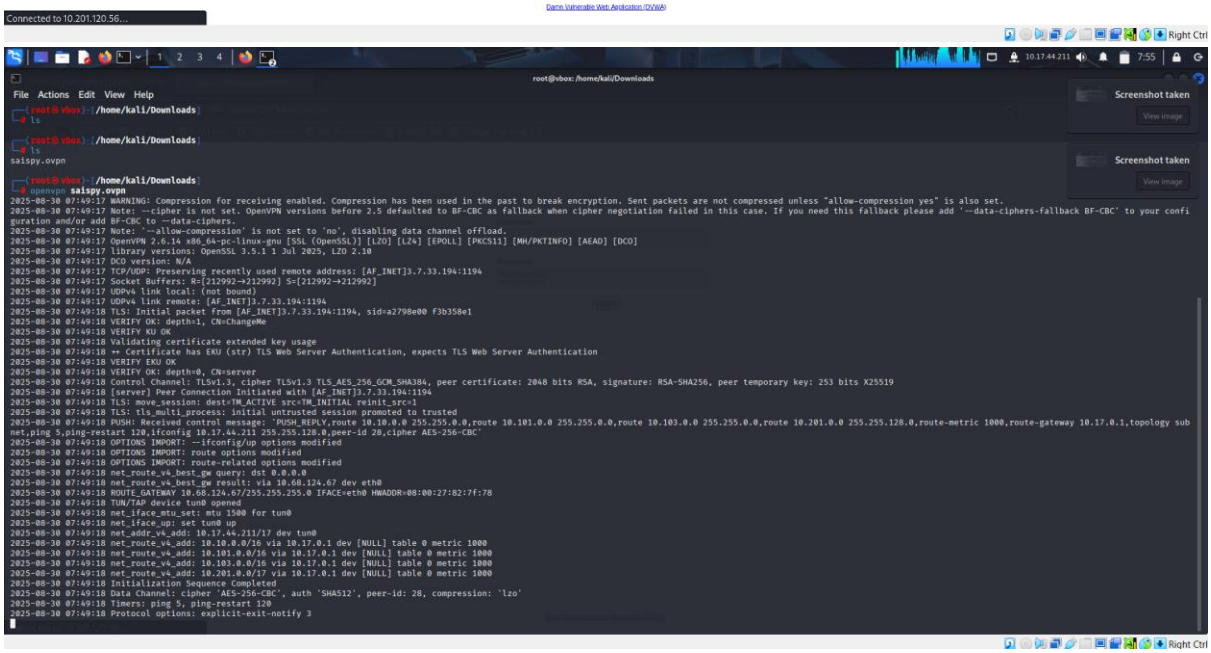
**2.2 Setup Steps**

1. **Logged into TryHackMe** account.
2. **Joined the "DVWA" Room** (or "VulnHub" equivalent depending on room name).
3. **Started the AttackBox** (or connected VPN if using own Kali VM).
4. **Launched the Target Machine** containing DVWA.
5. Accessed DVWA via the provided machine IP: http://10.201.120.56/login.php


Applications
Login :: Damn Vulnerable
https://tryhackme.com/room/dvwa
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Firefox prevented this site from opening 2 pop-up windows.
Preferences
Learn > DVWA
DVWA
Basic room for testing exploits against the Damn Vulnerable Web Application box
Easy 45 min
Share your achievement
Start Kali Linux
Save Room
892
Options
Damn Vulnerable Web Application
Room completed ( 100% )
Target Machine Information
Title
Target IP Address
Expires
DVWA
10.201.120.56
51min 59s
Add 1 hour
Terminate
Task 1 DVWA
DVWA is an awesome virtual machine commonly utilized in training and testing of new tools. This room is unguided and acts purely as a testing environment.
Start Machine
The credentials to login can easily be found online, but they are also included in the hint below, should you prefer to take the easy route.
Answer the questions below
Right Ctrl


TryHackMe | DVWA
Login :: Damn Vulnerable
10.201.120.56/login.php
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB


Username
admin
Password
Login


Connected to 10.201.120.56...
Damn Vulnerable Web Application (DVWA)
Right Ctrl
root@vbox: /home/kali/Downloads
File Actions Edit View Help
/home/kali/Downloads
ls
saispy.ovpn
openvpn saispy.ovpn
2025-08-30 07:49:17 WARNING: Compression for receiving enabled. Compression has been used in the past to break encryption. Sent packets are not compressed unless "allow-compression yes" is also set.
2025-08-30 07:49:17 Note: --cipher is not set. OpenVPN versions before 2.5 defaulted to BF-CBC as fallback when cipher negotiation failed in this case. If you need this fallback please add '--data-ciphers-fallback BF-CBC' to your configuration and/or add BF-CBC to --data-ciphers.
2025-08-30 07:49:17 Note: 'allow-compression' is not set to 'no', disabling data channel offload.
2025-08-30 07:49:17 OpenVPN 2.6.14 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
2025-08-30 07:49:17 library versions: OpenSSL 3.5.1 1 Jul 2025, LZO 2.10
2025-08-30 07:49:17 DCO version: N/A
2025-08-30 07:49:17 TCP/UDP: Preserving recently used remote address: [AF_INET]3.7.33.194:1194
2025-08-30 07:49:17 Socket Buffers: R=[212992->212992] S=[212992->212992]
2025-08-30 07:49:17 UDPv4 link local: (not bound)
2025-08-30 07:49:17 UDPv4 link remote: [AF_INET]3.7.33.194:1194
2025-08-30 07:49:18 TLS: Initial packet from [AF_INET]3.7.33.194:1194, sid=a2798e00 f3b358e1
2025-08-30 07:49:18 VERIFY OK: depth=1, CN=ChangeMe
2025-08-30 07:49:18 VERIFY KU OK
2025-08-30 07:49:18 Validating certificate extended key usage
2025-08-30 07:49:18 ++ Certificate has EKU (str) TLS Web Server Authentication, expects TLS Web Server Authentication
2025-08-30 07:49:18 VERIFY EKU OK
2025-08-30 07:49:18 VERIFY OK: depth=0, CN=server
2025-08-30 07:49:18 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 2048 bits RSA, signature: RSA-SHA256, peer temporary key: 253 bits X25519
2025-08-30 07:49:18 [server] Peer Connection Initiated with [AF_INET]3.7.33.194:1194
2025-08-30 07:49:18 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2025-08-30 07:49:18 TLS: tls_multi_process: initial untrusted session promoted to trusted
2025-08-30 07:49:18 PUSH: Received control message: 'PUSH_REPLY,route 10.10.0.0 255.255.0.0,route 10.101.0.0 255.255.0.0,route 10.103.0.0 255.255.0.0,route 10.201.0.0 255.255.128.0,route-metric 1000,route-gateway 10.17.0.1,topology subnet,ping 5,ping-restart 120,ifconfig 10.17.44.211 255.255.128.0,peer-id 28,cipher AES-256-CBC'
2025-08-30 07:49:18 OPTIONS IMPORT: --ifconfig/up options modified
2025-08-30 07:49:18 OPTIONS IMPORT: route options modified
2025-08-30 07:49:18 OPTIONS IMPORT: route-related options modified
2025-08-30 07:49:18 net_route_v4_best_gw query: dst 0.0.0.0
2025-08-30 07:49:18 net_route_v4_best_gw result: via 10.68.124.67 dev eth0
2025-08-30 07:49:18 ROUTE_GATEWAY 10.68.124.67/255.255.255.0 IFACE=eth0 HWADDR=08:00:27:82:7f:78
2025-08-30 07:49:18 TUN/TAP device tun0 opened
2025-08-30 07:49:18 net_iface_mtu_set: mtu 1500 for tun0
2025-08-30 07:49:18 net_iface_up: set tun0 up
2025-08-30 07:49:18 net_addr_v4_add: 10.17.44.211/17 dev tun0
2025-08-30 07:49:18 net_route_v4_add: 10.10.0.0/16 via 10.17.0.1 dev [NULL] table 0 metric 1000
2025-08-30 07:49:18 net_route_v4_add: 10.101.0.0/16 via 10.17.0.1 dev [NULL] table 0 metric 1000
2025-08-30 07:49:18 net_route_v4_add: 10.103.0.0/16 via 10.17.0.1 dev [NULL] table 0 metric 1000
2025-08-30 07:49:18 net_route_v4_add: 10.201.0.0/17 via 10.17.0.1 dev [NULL] table 0 metric 1000
2025-08-30 07:49:18 Initialization Sequence Completed
2025-08-30 07:49:18 Data Channel: cipher 'AES-256-CBC', auth 'SHA512', peer-id: 28, compression: 'lzo'
2025-08-30 07:49:18 Timers: ping 5, ping-restart 120
2025-08-30 07:49:18 Protocol options: explicit-exit-notify 3
Screenshot taken
Screenshot taken
Right Ctrl

# DVWA – 5 Vulnerabilities

## 1. Brute Force : OWASP: A07 – Identification & Authentication Failures

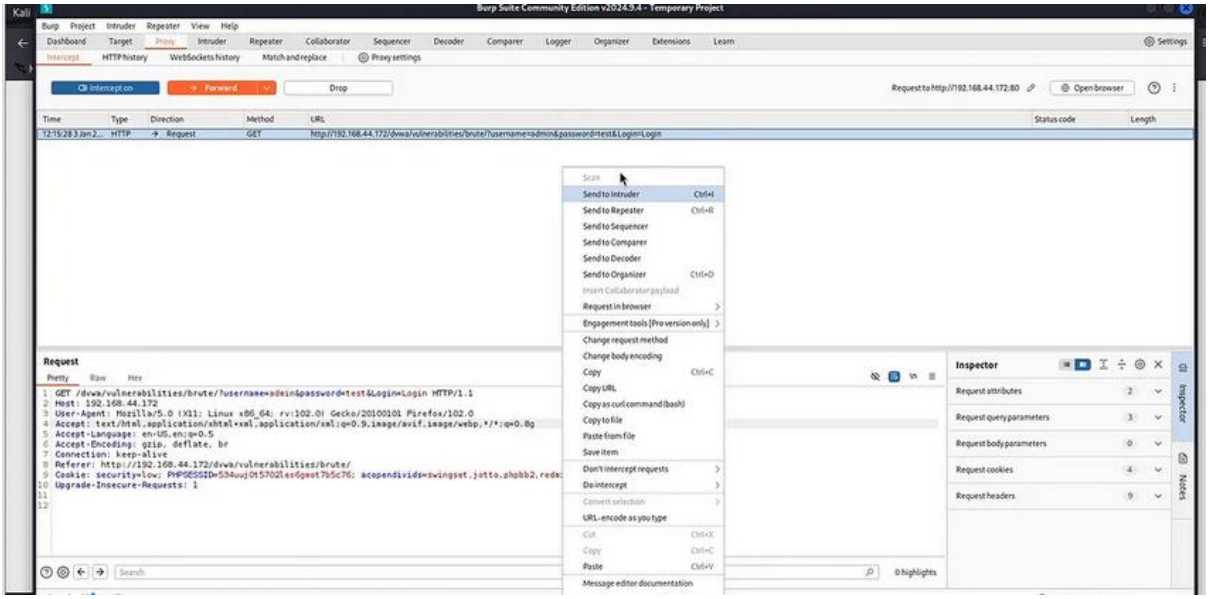
### Steps

1. DVWA → Brute Force.
2. Manually try 1–2 guesses to observe success vs. failure responses (status code, message, or timing).
3. Use a tool (e.g., Burp Intruder) only in this lab to iterate a small, harmless wordlist for username/password.
4. Watch for a distinct success response (e.g., different page content or status code) indicating a valid credential.

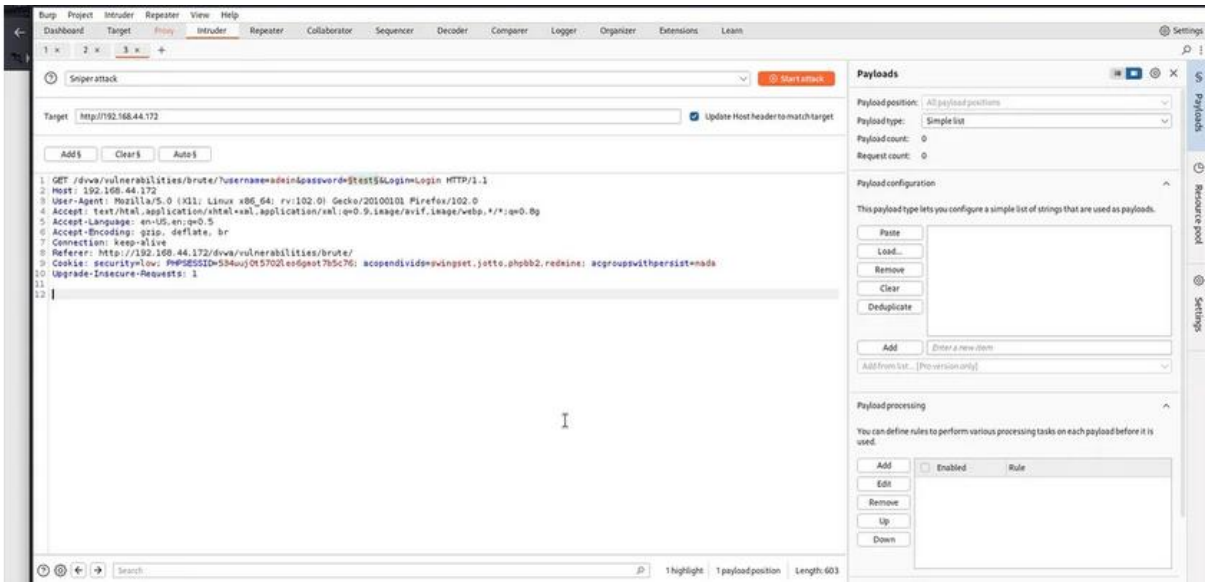
**Brute Force Source**

```php
<?php

if( isset( $_GET['Login'] ) ) {

    $user = $_GET['username'];

    $pass = $_GET['password'];
    $pass = md5($pass);

    $qry = "SELECT * FROM `users` WHERE user='$user' AND password='$pass';";
    $result = mysql_query( $qry ) or die( '<pre>' . mysql_error() . '</pre>' );

    if( $result && mysql_num_rows( $result ) == 1 ) {
        // Get users details
        $i=0; // Bug fix.
        $avatar = mysql_result( $result, $i, "avatar" );

        // Login Successful
        echo "<p>Welcome to the password protected area " . $user . "</p>";
        echo '<img src="' . $avatar . '" />';
    } else {
        //Login failed
        echo "<pre><br>Username and/or password incorrect.</pre>";
    }

    mysql_close();
}

?>
```

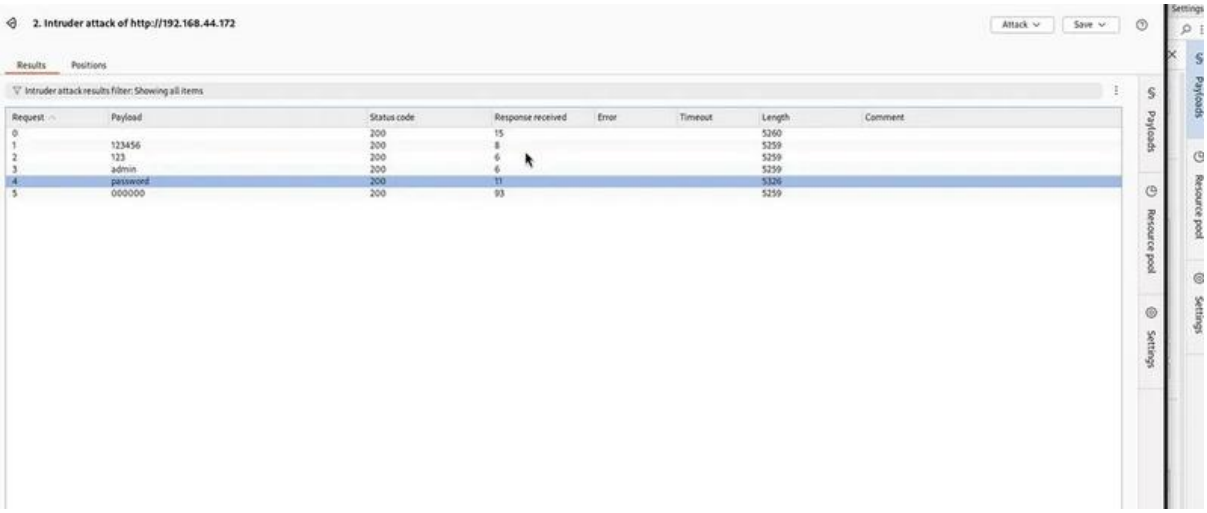
### 1.Analysis of source Code to understand working of login



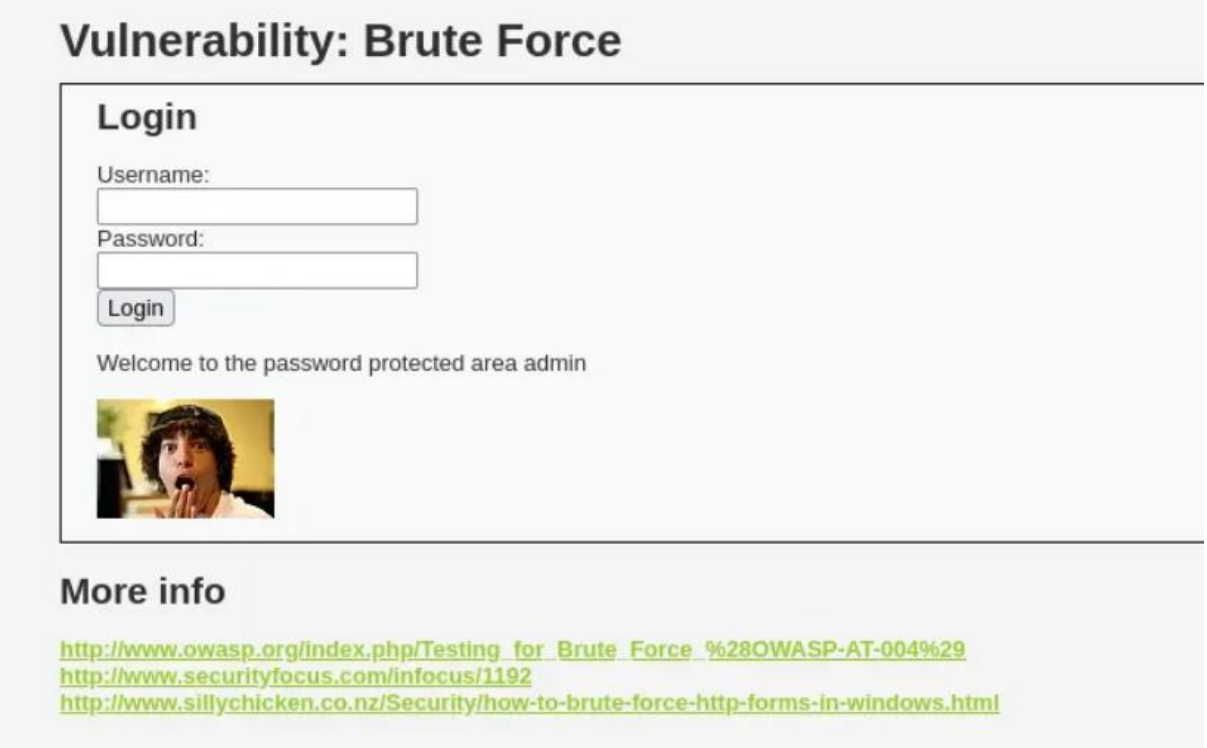

## 2.Intercept a login request using burpsuite




## 3.sending request to Intruder and positions to add password




**4.start attack and check output**

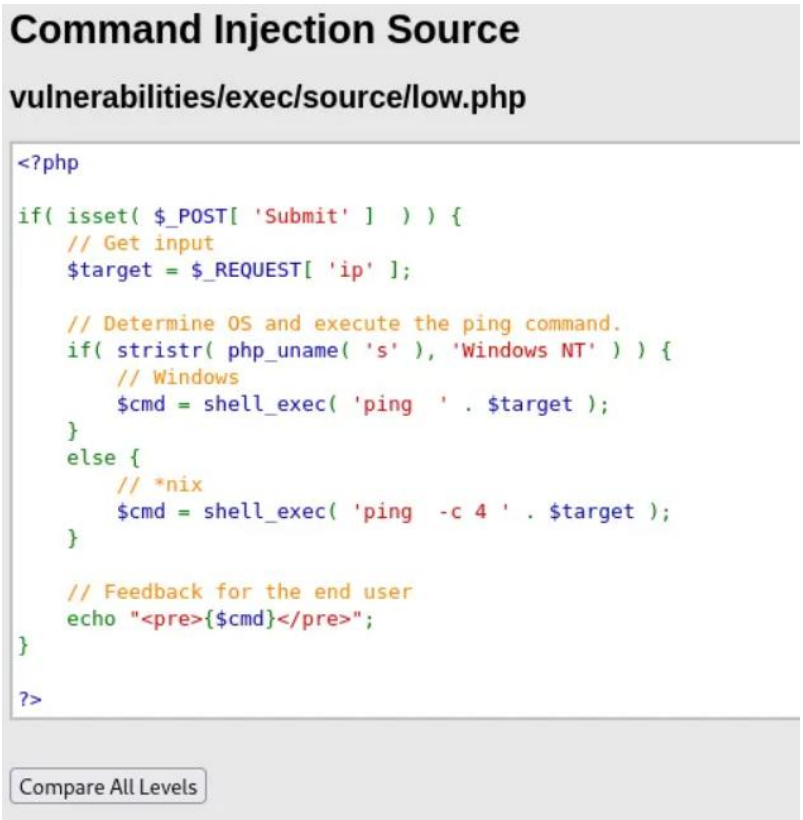



**5.after trying multiple passwords we got correct password and we get admin access**

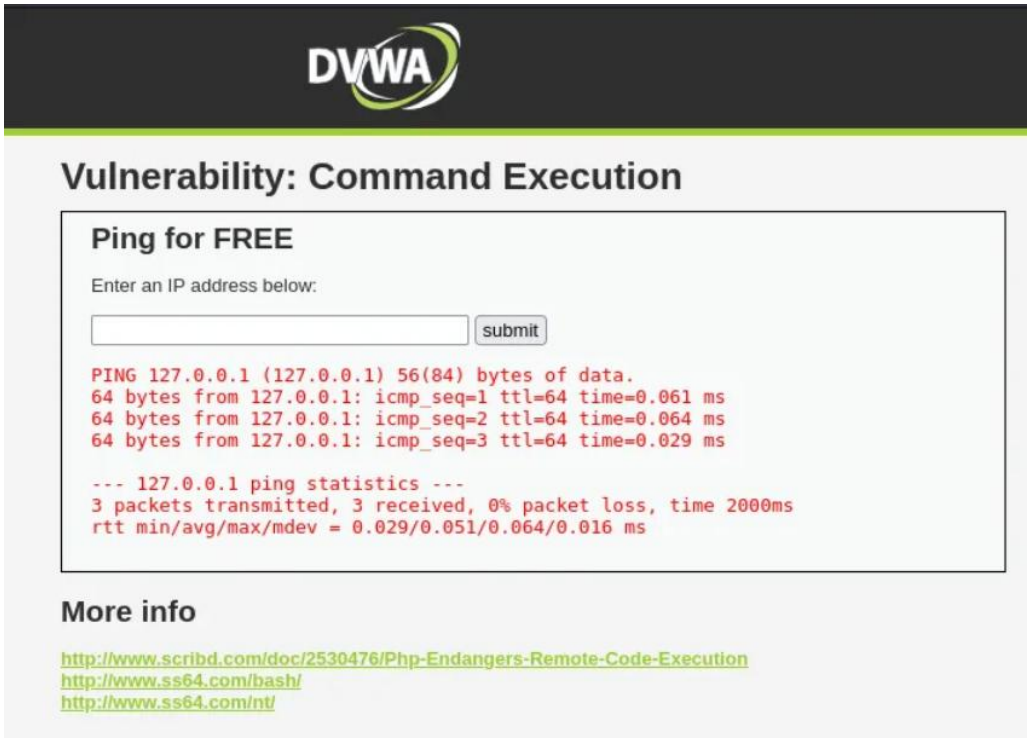
## Command Injection: OWASP: A03 – Injection

Steps

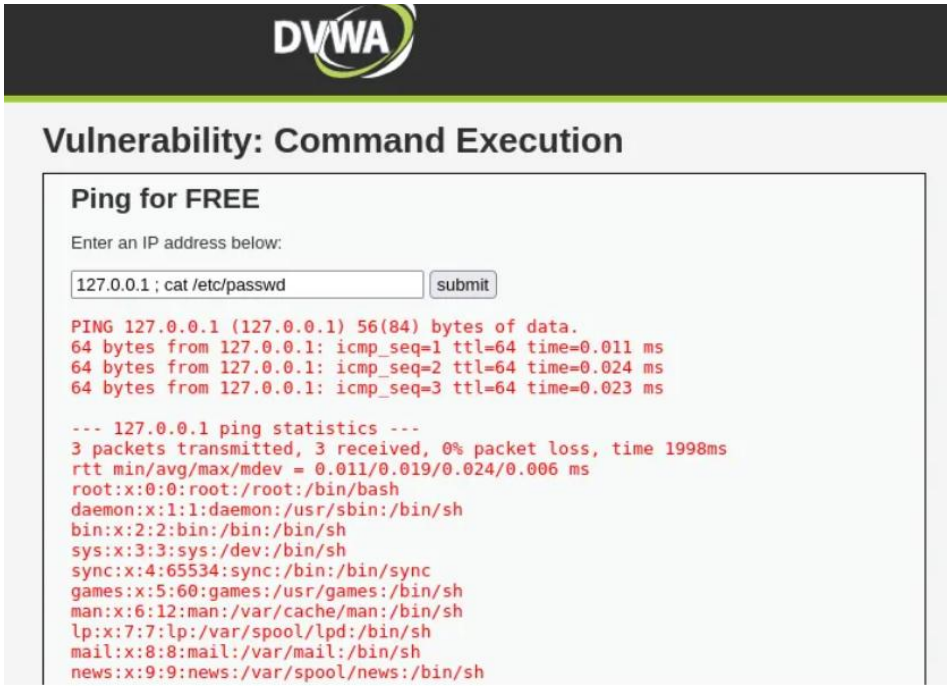
1. DVWA → Command Injection.
2. Enter a normal IP (e.g., 127.0.0.1) and run it to see the baseline ping output.
3. Now attempt to append a second command using a common command separator (e.g., ; or &&) followed by a harmless system command (e.g., printing the current directory or user).
    - Example pattern to try: 127.0.0.1 [separator]
4. If successful, the output area will contain both the ping output and the second command's output.




## 1.Analyzing Source Code




**2.ping 127.0.0.1 to understand working of command**




**3. 127.0.0.1 ; cat /etc/passwd to extract a password without authentication**

# SQL injection: OWASP: A03 – Injection

Steps

1. DVWA → SQL Injection.
2. In the user ID input, first test input handling: enter a single quote (just a ') to see if an error appears (e.g., SQL syntax error).
3. Try a tautology pattern (replace with your own equivalent), e.g. something that makes the WHERE clause always true.
4. Observe whether the application returns additional rows or bypasses filtering.
5. Use Burp Repeater (optional) to replay the same request with small variations and note server responses.

**What is SQL Injection ?**

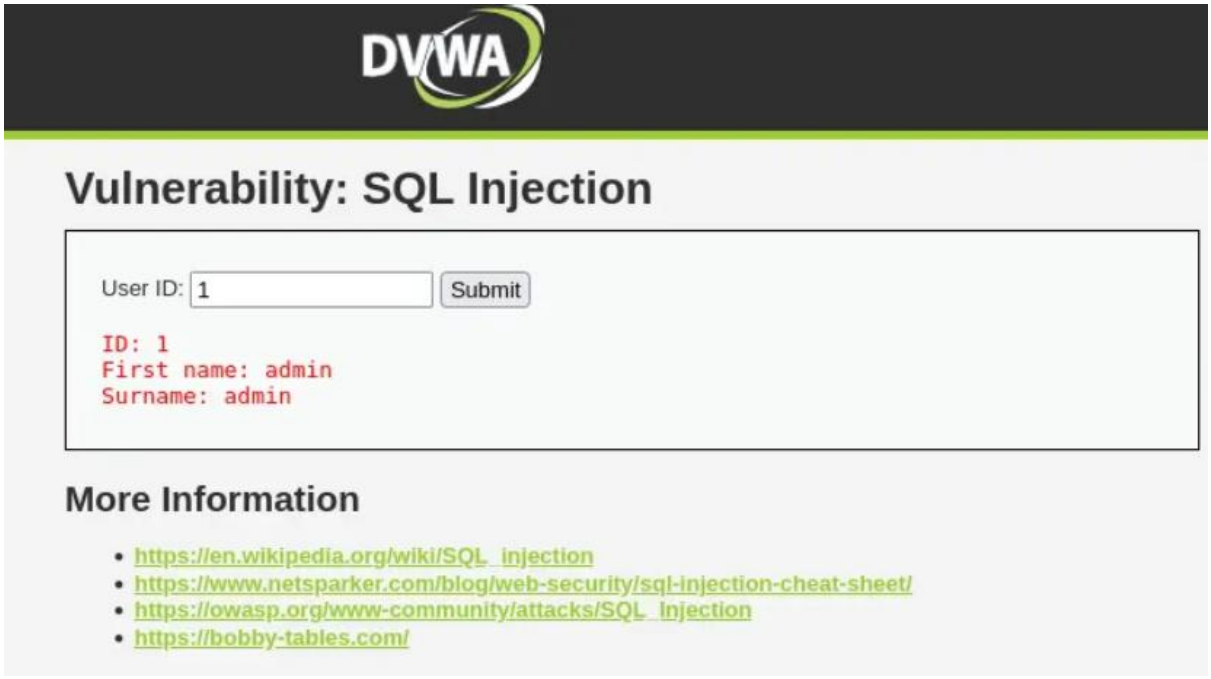
SQL injection is a technique used to manipulate SQL queries, allowing attackers to access, modify, or delete data in a database by exploiting vulnerable input fields .

**SQL Injection Source**

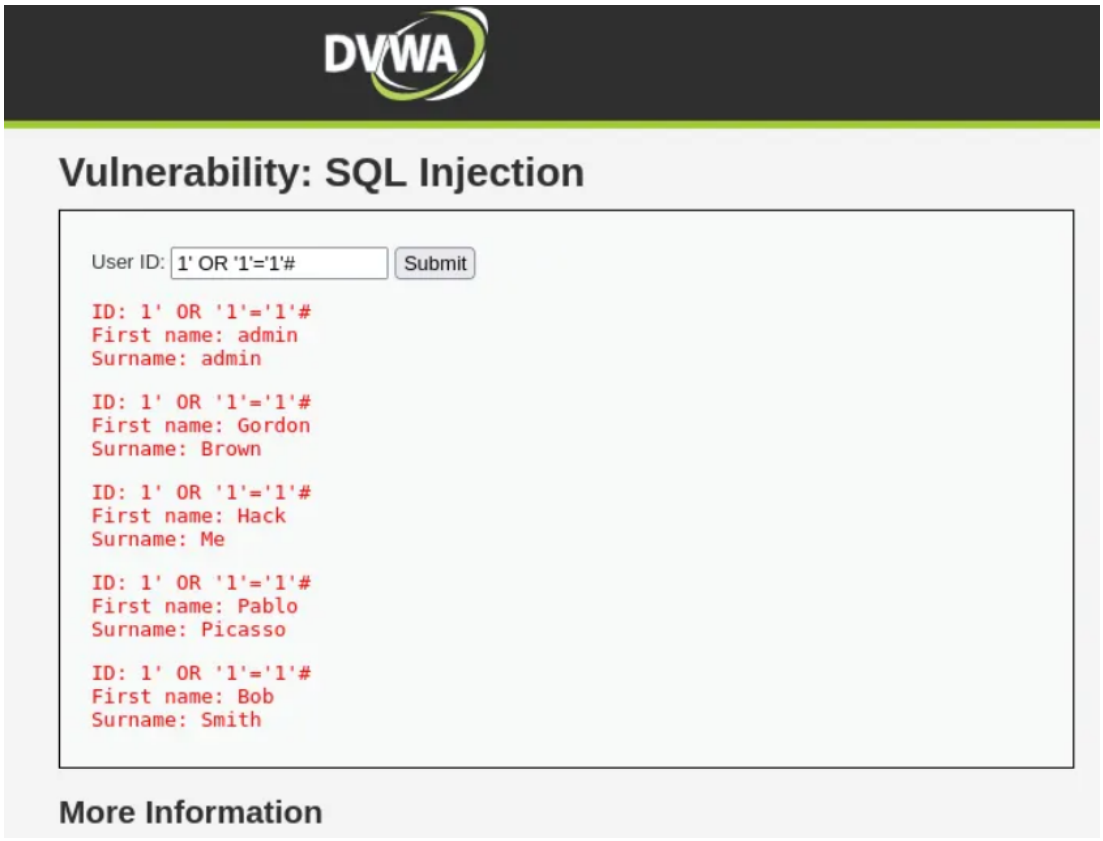
**vulnerabilities/sqli/source/low.php**

```php
<?php

if( isset( $_REQUEST[ 'Submit' ] ) ) {
    // Get input
    $id = $_REQUEST[ 'id' ];

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            // Check database
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
            $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS["___mysqli_ston"]) : (($_
pre>' );

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Get values
                $first = $row["first_name"];
                $last  = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
            }

            mysqli_close($GLOBALS["___mysqli_ston"]);
            break;
        case SQLITE:
            global $sqlite_db_connection;

            #$sqlite_db_connection = new SQLite3($_DVWA['SQLITE_DB']);
            #$sqlite_db_connection->enableExceptions(true);

            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
            #print $query;
            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                echo 'Caught exception: ' . $e->getMessage();
                exit();
            }
```

# 1. Analyzing Source Code




**2. For example, if we enter the ID 1, the code fetches the first name admin and last name admin, which are associated with that ID.**




**1' OR '1'='1'# inject a payload**

## XSS: A03 – Injection (XSS)

XSS is a technique in which attackers inject malicious scripts into a target website and may allow them to gain access control of the website. If a website allows users to input data like comment, username field and email address field without controls then attacker can insert malicious code script as well.

TYPES OF XSS:

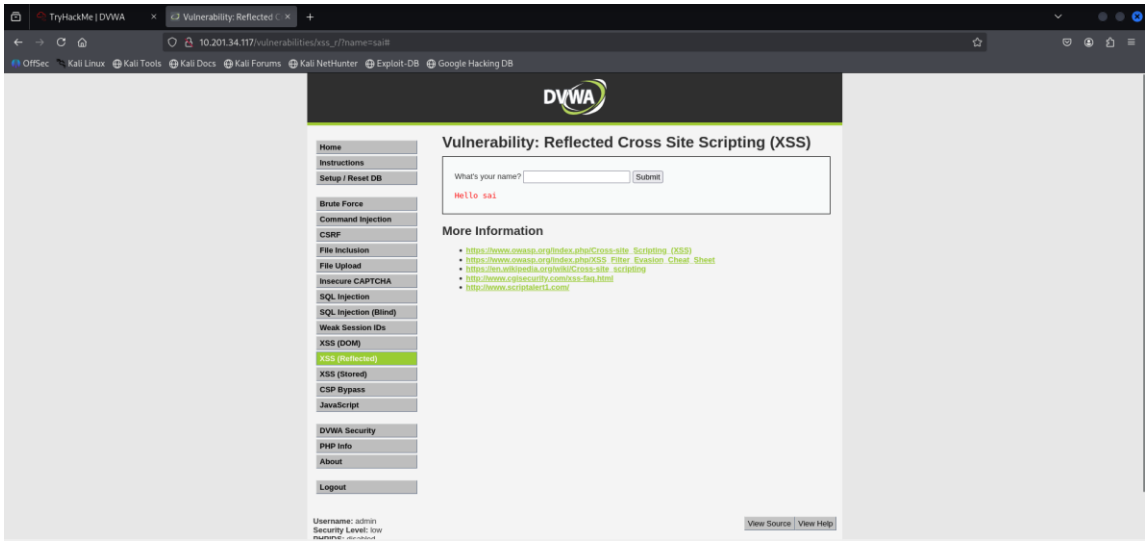
1. Reflected XSS

2. Stored XSS

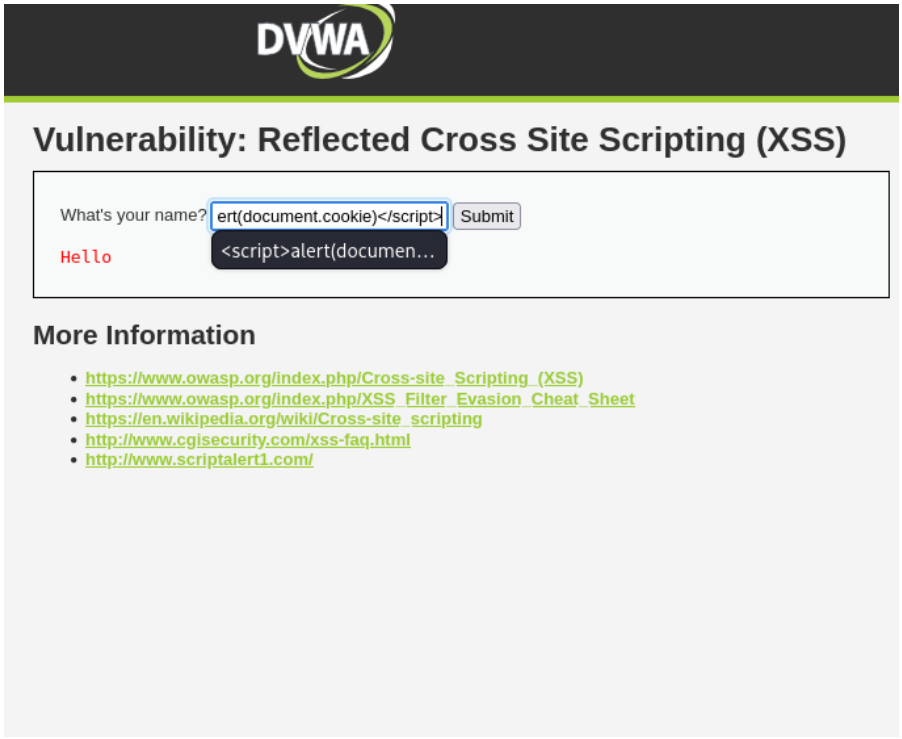
3. Dom Base XSS

### Steps

1. DVWA → XSS (Stored).
2. In the message/comment field, submit a benign marker first (e.g., TEST123) to confirm storage.
3. Now submit a harmless script payload that proves execution (e.g., a minimal alert or DOM-writing snippet).
4. Reload/return to the page to see the payload executing from stored data.

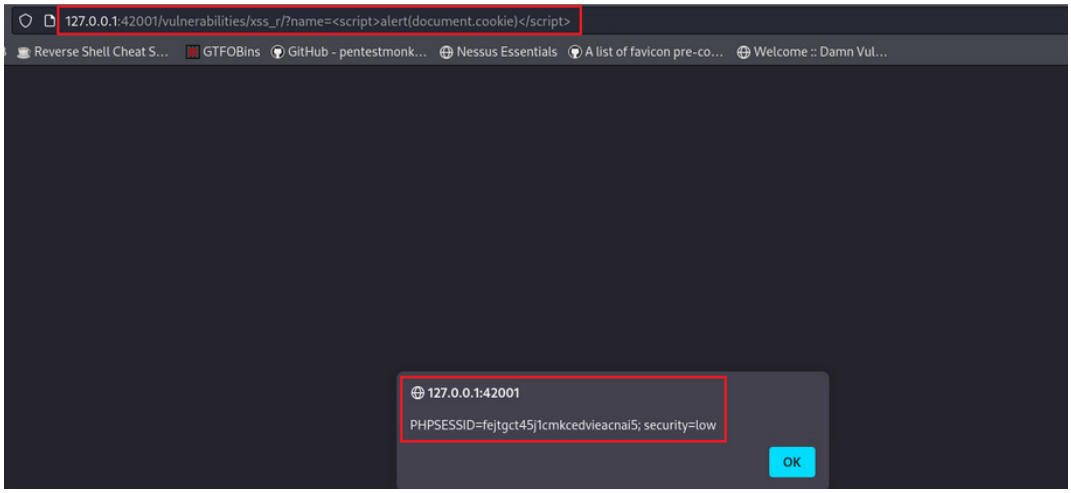


**1.Putting our name in the input form, we notice the the parameter name appears on the address bar:**




**2.Instead of our name, we can try a simple payload to see it will work:**

**<script>alert(document.cookie)</script>**




**3.script is successfully executed**

# CSRF: Cross-Site Request Forgery

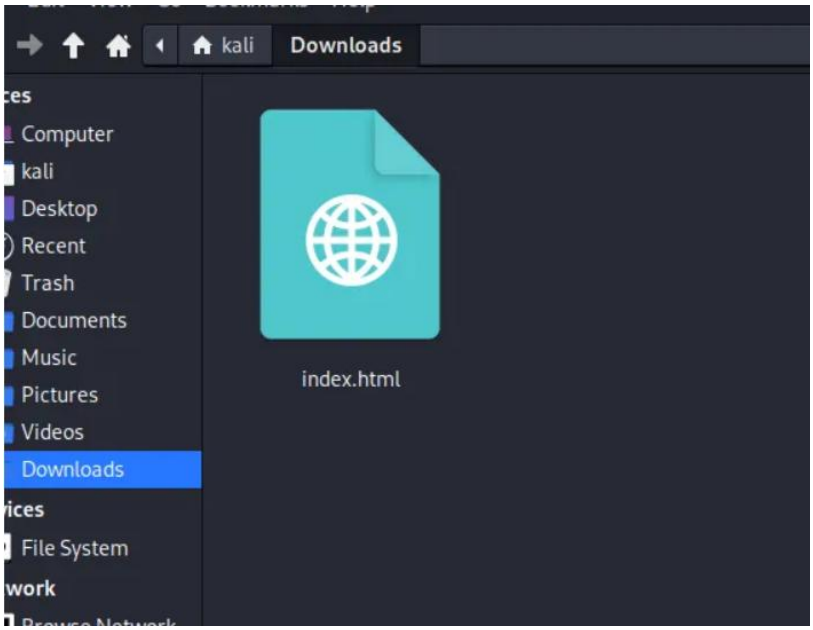
CSRF, which stands for Cross-Site Request Forgery, is a type of attack where someone takes advantage of a user's active session on a website to make them unintentionally perform actions they didn't intend to. This attack works when the user is already logged into the website or application.


## 1.Analysis of source Code

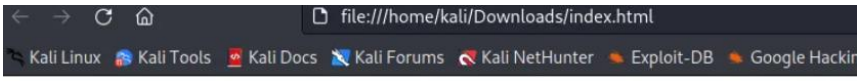

## I will Create a new password "123" and click on Change

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h3>Click to Download Fifa 2023</h3>
    <a href="http://172.17.0.1:8888/vulnerabilities/csrf/?password_new=12345&
    password_conf=12345&Change=Change#">Fifa 2023</a>
</body>
</html>
```

Now we will Display the HTML code for the page, which includes a link to download a game called “FIFA 2023. and password has been changed by attacker”



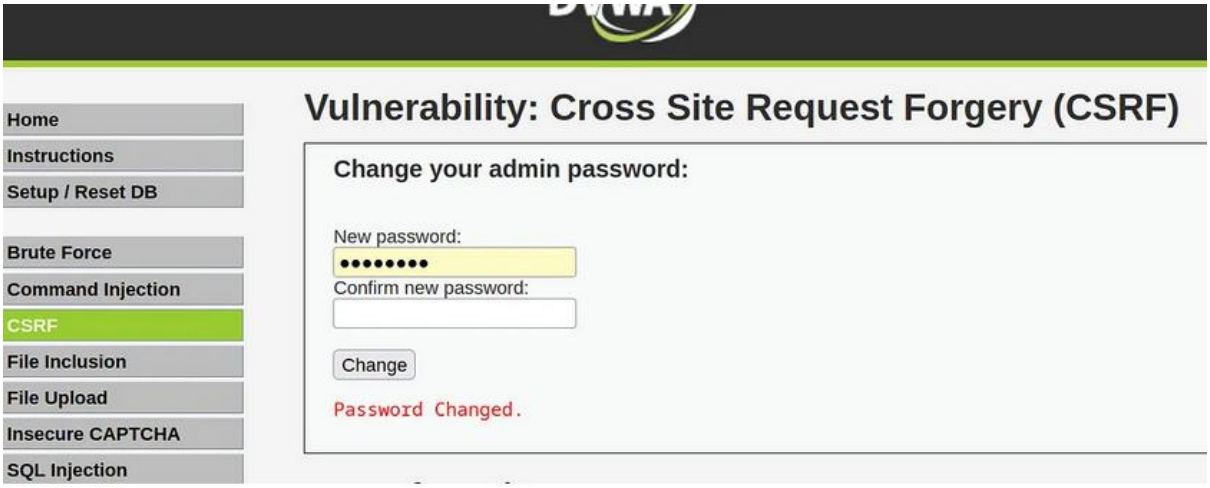

If the victim tries to open the html page. It will looks like this….




# Click to Download Fifa 2023

Fifa 2023

When victim tries to click on the FIFA link, the password “12345” will be changed automatically




**We can see that password has been changed**